Pagination

Prof. Dr. Peter Braun

6. Dezember 2024

1 Titlepage

Pagination

Prof. Dr. Peter Braun

thws Technical University of Applied Sciences Würzburg-Schweinfurt

No Text – maybe some music :-)

2 Learning Goals

Learning Goals

- Pagination is a technique to handle large collections
- Three approaches: Page, Offset/Size, and Cursor
- Inconsistencies when using pagination
- Applying the hypermedia principle to pagination

Welcome to this unit on the topic of pagination. You have already heard a lot about the implementation of Web APIs in the last units and have already gained some experience implementing a backend. Today and in the subsequent units, we will discuss some special topics. We will start with the issue of pagination, which is about handling many resources of one type. We will present three approaches to implementing a backend that allows the client to load only small subsets of a large result set. However, when using such procedures, a problem may arise if other clients work on the data in parallel, specifically adding new records or deleting existing records. We will explain these two problem cases and suggest solutions for them. This will be the first unit in which we will introduce the hypermedia concept. This concept means that the backend includes hyperlinks in the response to let clients continue with the next request as easily as possible. This hypermedia principle is the major difference between Web APIs and REST APIs.

3 Problem with Large Collection Resources

Problem with Large Collection Resources

- Collection resources can become very large
- There is no need for clients to load all data at once
- Clients should load only small sub-sets or **pages**

First, let's briefly discuss why we must deal with pagination. The amount of resources, i.e., records of a type, can become very large in a real-world application. Imagine a social network like Facebook or Instagram, for example. The number of users registered there is greater than 3 billion. But even in smaller software systems, such as those used in a university environment, we are talking about more than 9,000 students or thousands of exams to be taken each semester. So far, we have always implemented accessing the set of all resources so that all objects or records are returned to the client. However, we have already discussed one method to reduce the result set. We used very simple filter criteria to search for specific records. This also reduces the number of items in the response message. In the next unit, we will extend this filter criteria. However, if we assume that we do not want to use a filter, then the client, such as a mobile or web application, would have to be able to cope with large result sets. However, we have two problems here. First, provisioning large result sets in the server takes a very long time due to database access and conversion to a representation format, and a lot of data must be transferred. On the other hand, the client must also have enough free memory to handle large result sets. The crucial point is that transferring large sets of results is entirely pointless in most cases. At least when the results are displayed to a human user in a list, the space on the screen is limited, and humans cannot deal with large result sets in this way. Today, we will present several technical approaches to the solution, which divides the result into smaller subsets, also called pages. Hence, the name pagination. The server always responds to requests with only a subset of the entire result set, such as ten or twenty elements. If the client wants to see more results, he must load the next page with a separate request from the server. You all know these procedures from dealing with search engines, for example, when only the first ten or 20 hits are displayed. If you want to see more results, you

must press a button. With mobile apps, a pattern has become more established where you scroll down the results list, and as soon as you reach the end of the current page, the app automatically loads the next page.

4 Approach 1: Pagination Using a `Page` Parameter

Approach 1: Pagination Using a `Page` Parameter

`http://localhost:8080/demo/api/persons?page=3`

- Clients request a specific page number
- Counting usually starts at `1` (default value)
- The server defines the page size

Today, we will present three pagination procedures. In the first method, we use a query parameter called Page. Please take a look at the example of a URI next to me. We access the collection of all resources of type Person, append the parameter Page to the address after the question mark, and pass the value three. Thus, we want to load the third page of the result set. This procedure is straightforward for the client because it only needs to know which page of the result set it wants to load. However, the process is also somewhat inflexible because the client does not influence the result set size. This size is determined exclusively by the server or by the server's developer. A single page of the result set should typically contain ten or twenty elements, but this depends very much on the specific use case. The page numbering typically starts with one. This information should also be shared with the client in advance, but we will see later that this doesn't matter. The numbering could also start with zero, and the client would not notice if we used the hypermedia principle correctly. We will look at that later.

5 Implementation of a page Parameter

Implementation of a page Parameter

```
1  @Path( "/persons" )
2  public class PersonService
3  {
4          @GET
5          @Produces( MediaType.APPLICATION_JSON )
6          public Response getPersons(
7             @DefaultValue( "1" ) @QueryParam( "page" ) int pageNumber )
8          {
9                  List<Person> allPersons = this.personDAO.loadAll();
10                 List<Person> page = Pagination.page( allPersons,
                       pageNumber );
11                 return Response.ok( page ).build();
12         }
13 }
```

- But: Loading all elements from the database is too expensive – **don't do this**

We have already learned to handle query parameters in REST Easy in several places, so the source code you see next to me should not surprise you. In line 7, the parameter "Page Number" is annotated with the name "Page" so that REST Easy knows what to do with the query parameter in the address. Within the method, we first load all elements of type Person from the database in line 9 and cut out the desired page from this result set in line 10. In line 11, we send only the requested subset, i.e., the desired page, back to the client as part of the response message. This source code is for demonstration purposes only; don't implement this in reality. The next slide will show you how this should be done using a real database system.

6 Delegating Pagination to the Database

Delegating Pagination to the Database

```
1  @Path( "/persons" )
2  public class PersonService
3  {
4    @GET
5    @Produces( MediaType.APPLICATION_JSON )
6    public Response getPersons(
7        @DefaultValue( "1" ) @QueryParam( "page" ) int pageNumber )
8    {
9     List<Person> page = this.personDAO.loadAll( pageNumber, 20 );
10    return Response.ok( page ).build();
11   }
12 }
```

- Pagination in SQL: keywords `OFFSET` AND `LIMIT`
- For example: `OFFSET (page-1)*20 LIMIT 20`

Before we come to the next procedure, I would like to show you briefly how pagination would be implemented in reality. Initially, we argued that result sets could become very large, so it only makes sense to load some results from the database into the server's main memory and then extract a page with ten or 20 elements. But this is precisely what we had done in our first example. In reality, you would pass the subdivision into the pages down to the database level. The method "loadAll" in line 9 in the source code next to me now gets two parameters. One is the number of the desired page, and the other is the size of the page. At the database level, we can implement pagination using, for example, the SQL keywords Offset and Limit. The value for Offset specifies how many elements at the beginning of the result set should be skipped until the beginning of the desired page. The value for Limit specifies the page size. If we start with the numbering of the pages at one and limit the page size to 20 elements, we could work with the information in the last line next to me, for example.

7 Approach 2: Pagination Using Offset and Size

Approach 2: Pagination Using Offset and Size

`http://localhost:8080/demo/api/persons?offset=0&size=20`

- Clients request a sub-set of all results
- Parameter `offset` defines the start index (default `0`)
- Parameter `size` defines the number of elements (default for example `20`)

These two SQL keywords, offset and limit, lead us to our second approach, which can be used in the backend for pagination. In this approach, exactly these two parameters for offset and limit are used as query parameters. Again, please look at the example URI next to me first. We query the set of all resources of type Person and specify the parameter offset value 0 and the parameter size value 20. Instead of the SQL keyword Limit, we use the query parameter size. But the meaning is the same. With this approach, the client can very finely control which subset of the total result set it wants to see. In particular, it can use the size parameter to control how large the result set should be, and it could thus react to different screen sizes, for example. Usually, more results can be displayed on one page on a website or a tablet than on a smartphone. If these two parameters are omitted, the server does not respond with the entire result set but has default values for both parameters. So, if the client does not specify an offset parameter, the server responds with the beginning of the result set, so the offset is practically 0. If the client does not specify a value for size, the server must again have stored a default size for the page for this case, for example, 20 elements. By the way, it also makes sense that the server does not accept all values for size. The client could override the idea of pagination by simply specifying the largest allowed value for the integer data type for the size parameter. The server must control this size value and should have a defined maximum page size in addition to a default value for the page size.

8 Implementation of `offset` and `size`

Implementation of `offset` and `size`

```
1  @Path( "/persons" )
2  public class PersonService
3  {
4          @GET
5          @Produces( MediaType.APPLICATION_JSON )
6          public Response getPersons(
7              @DefaultValue( "0" ) @QueryParam("offset") int offset,
8              @DefaultValue( "20" ) @QueryParam( "size" ) int size )
9          {
10             // pagination is done by the database
11             return Response.ok( page ).build();
12         }
13 }
```

Here, we will first look at a possible implementation as an example to show you what the method's signature could look like. We have two parameters in lines 7 and 8. You can see the default values for these parameters on both lines. We then load all results from the database again and return the requested part.

9 Open Questions

Open Questions

- Which pagination approach was chosen?
- What are the default values?
- How does the client navigate to the next page?

So, you have been introduced to two of the three processes, and we would like to take a short break to clarify any unanswered questions. So, there are at least two methods for pagination, and the server can choose one of these methods for its implementation. The client must be informed about this decision so the developer can incorporate this method accordingly. However, this would lead to some coupling between client and server. Later, the client must also be changed when the paging mechanism should be changed on the server. So, the question is: is there any better solution to inform the client about the paging mechanism? There is also the question of which default values the server uses. The numbering of the page typically starts with 1 in the page approach, but the starting value for the offset in the offset size approach is 0. How does the client know this? And more importantly, does the client need to know this at all? Lastly, how does the client know there is a subsequent page with more results, and what address should be used for that? Let's look at the answers now.

10 Clients Should Not Know the Pagination Approach

Clients Should Not Know the Pagination Approach

- `/demo/api/persons`
- `/demo/api/persons?offset=0&size=20`

The first question was how the client knew which method the server had chosen. The straightforward answer is that it doesn't need to know. All three methods we present today can be implemented so that it is entirely transparent to the client which method has been used. To realize this, we have to extend the server implementation a bit. We have already completed the first step using default values. When the client accesses the upper of the two Uris next to me, the default values we set for pagination will take effect depending on our chosen procedure. If we have implemented the offset and size method and set the default values for offset to zero and size to 20, then the content of the second URI is precisely the same as the first. The client knows the first of the two Uris; it has received this Uri from the backend or as a link in the response message to a previous request.

11 Navigation From Page to Page

Navigation From Page to Page

- Relation type `self`: the URI used in the request
- Relation type `prev`: Hyperlink to previous page
- Relation type `next`: Hyperlink to next page

- If there is no previous page, don't create a link

- **This is an example of the hypermedia principle**

How does the client know which URI to use to access the next or previous page of the result set? One naive approach is for the server to reveal information about the selected pagination approach. If the client knows the offset/size approach is used, the values can be set by choice. At this point, I want to introduce the hypermedia principle. This is now very important. According to the hypermedia principle, clients should only use URLs that they have received from the backend. With only one exception: the first URL that clients need to send the first request to the backend is hard-coded into the client. Clients should not be necessary to create or assemble URLs by themselves. The hypermedia principle states that the backend always provides the client with precise URLs that the client should use next. We can use this principle wonderfully in pagination. In the response message's header, the server sends the client exactly those URIs as links that must be used to access the next or the previous result page. Again, there are standards for these two links, and the relation names must be "next" for the next and "prev" for the previous page. The relation name "self" is also specified, by the way. For the client, it is completely transparent which "pagination" approach the server uses. The client only works the hyperlinks they received under the given relation names "next," "prev," and "self." Of course, the server would not deliver a link to the next page if the client has already requested the last page of the result set. Similarly, no link to a previous page is returned if the client has just loaded the first page of the result set.

12 Return Hyperlinks to Previous and Next Pages

Return Hyperlinks to Previous and Next Pages

```
1  @Path( "/persons" )
2  public class PersonService {
3          @GET
4          @Produces( MediaType.APPLICATION_JSON )
5          public Response getPersons(
6             @DefaultValue( "20" ) @QueryParam( "size" ) int size,
7             @DefaultValue( "0" ) @QueryParam( "offset" ) int offset )
8          {
9             List<Person> page = this.personDAO.loadAll( offset, size );
10            URI thisPage = Pagination.selfUri( offset, size );
11            URI previousPage = Pagination.previousPage( offset, size );
12            URI nextPage = Pagination.nextPage( offset, size );
13            return Response.ok( page )
14                           .link( previousPage, "prev" )
15                           .link( nextPage, "next" )
16                           .link( thisPage, "self" )
17                           .build();
18         }
19 }
```

The slide beside me shows an example of implementing this. In line 9, we use a method call to load all data from the database. Again, this is only for demonstration and should not be done in practice. In lines 10 to 12, we use a class, “Pagination,” to create three hyperlinks that are later added to the response header in lines 14 to 16. In a real implementation, it is much more effort to create these three hyperlinks because you have to use information from the last SQL select statement to know the total number of results since this information is necessary to determine whether there is a next link.

13 Inconsistencies When Using Pagination

Inconsistencies When Using Pagination

Id	Resource	Position	Page
10	A	1	1
3	B	2	1
23	C	3	1
45	D	4	1
1	E	5	1
12	F	1	2
99	G	2	2

- What if a new resource is added?
- What if a resource is deleted?

If one client loads a large result set completely and another client subsequently changes the data, we don't see that at first. However, if the second client changes a record and we query that same record again later with a GET access, we get the updated data. If the other client deletes a record that is still in our result list, we notice this by the status code 404. If we load the data page by page instead of all data at once, we must be prepared for two other problems. We call both cases "inconsistencies;" the first refers to a client adding a record, and the second relates to a client deleting a record. To illustrate these two problems, we use a table that you see next to me. The table shows the resources abstractly, labeled with letters A, B, etc. The respective ID from the database is in the column left to this. You see the values 10, 3, and so on. The resources should be sorted, not by the IDs, as you can easily see, but by the content of the resources. The two right columns describe the position of this record and the result page on which it is contained. The page size is five, and when accessing the first page of the result set, the server would return resources A to E to the client. The remaining two resources, F and G, should be returned when accessing the second page.

14 Adding a New Resource

Adding a New Resource

- After resource `Da` was added:

Id	Resource	Position	Page
10	A	1	1
3	B	2	1
23	C	3	1
45	D	4	1
117	**Da**	**5**	**1**
1	E	1	2
12	F	2	2
99	G	3	2

- Resource `Da` is not seen by the client
- Resource `E` is again shown on page 2

Now consider the following situation: The client has loaded the first page of the result set with resources A to E and will load the second page next. But before it can trigger this second request, another client adds a new record that will appear as the last element of the first page because of the sorting of the resources. The new record is assigned ID 117 in the table next to me. After the other client adds this record, the first client triggers the request to the second page. Now, we see two things. First, the new record with ID 117 is not part of the second page; thus, the client would not see this new record. Secondly, the record with ID one and resource E, which was already transferred to the client as part of the first page, is now in the first position on the second page. So our client would get this record twice.

15 Deleting an Existing Row

Deleting an Existing Row

- After resource `A` was deleted:

Id	Resource	Position	Page
3	B	1	1
23	C	2	1
45	D	3	1
1	E	4	1
12	F	5	1
99	G	1	2

- Content `F` is never transmitted.

The second problem arises when another client deletes a record. Again, we assume our client has loaded the first page with results A to E. Then, another client deleted the record with ID ten and resource A, and our client requested the second page. In the table next to me, you can see that the record with ID twelve and resource F has suddenly moved up one position and would now be the last element on the first page. Now, when our client queries the second page, the record with ID 99 is the first one on the second page. Our client would never see the record with resource F. In the first request, it was not part of the first page, and in the second request, it is not part of the second page.

16 How to solve these problems?

How to solve these problems?

- It depends on your application and the use cases.
- Transmitting content twice is not a big problem.
- Missing content could cause big problems.

You could now argue that these two problems are not that serious. Perhaps you already have a concrete use case in mind and have considered what would happen if a client received data twice or did not receive a single data record. The probability of such problems depends very much on the concrete use case. We must consider whether we can accept these two problems or if we need a solution. These two issues are somewhat irrelevant for a resource that is rarely or never changed. Suppose we can assume that the client or end users are more likely to access the data via a search anyway and that filter criterion can generally make the result set so small that we never have to jump to a second page. In that case, we can also rather neglect the problem. Which of these two problems is more serious, by the way? That, too, depends very much on the use case, but in general, the missing transfer of expected data is less easy for the end-user to comprehend than the absent transfer of a data set that has just been added. After all, the time of access is already decisive here. Fortunately, however, there is a solution, which we will now look at as the third approach in today's unit.

17 Approach 3: Cursor-based Pagination

Approach 3: Cursor-based Pagination

- Use the content of the last element as separator.
- For example: `/api/persons?lastSeen=E`

The third approach uses a position marker or cursor. As part of the result set, the server transmits a link to the next page to the client, indicating what information the client has already received. The parameter name could be "last-seen," as in the example next to me. The value used here is the content of the last resource on the previous page, which in the example would be the resource with the content E. If the server processes GET access to this Uri, it sends the next page of the result set in the response message, beginning with the first data record that follows data record E. The client now needs all subsequent data records on the next page. The client has already received record E on the last page and requires all subsequent records on the next page.

18 Problem Solved?

Problem Solved?

- After resource `Da` was added:

Id	Resource	Position	Page
10	A	1	1
3	B	2	1
23	C	3	1
45	D	4	1
117	**Da**	**5**	**1**
1	E	6	1
12	F	1	2
99	G	1	2

- After resource `A` was deleted:

Id	Resource	Position	Page
3	B	1	1
23	C	2	1
45	D	3	1
1	E	4	1
12	F	1	2
99	G	2	2

- The first `GET` request returned resources `A` to `E`
- The `next` link in the response is `/api/persons?lastSeen=E`

Let's look at the two problems and whether we have found a solution with the position marker approach. Our client has received the resources A to E with the first GET request and for the second accesses the Uri, which is at the very bottom of the slide. The server would first look at the entire result set and transfer the resources with ID twelve and the content F on the second page. We do not have a double transfer of the record with content E as in the other two procedures. However, the record with ID 117, which another client added between our two GET accesses, will still need to be visible to us. But we can't solve that problem in the general case. The new record could also be at a much earlier place in the result set, for example, in the beginning, and we would not notice this during the page-by-page processing of the result set. You can see the situation in the right column after deleting the resources with the ID ten and the content A. Our client has received the data A to E in its first request and would also correctly receive the data set with the ID twelve and the content F as the first with its second request. We have also solved this problem; our client now sees an F data set.

19 Implementation of the Cursor-based Approach

Implementation of the Cursor-based Approach

- Example to get the next page based on the id:

```
1 SELECT *
2   FROM some_table
3 WHERE filter_criteria
4     AND id > ?
5 ORDER BY id ASC
6 LIMIT 20
```

- If the result is ordered by some other column, you have to combine this with the id:

```
1 SELECT * FROM ... WHERE (col, id) > (?,?) ORDER BY col, id ASC
```

- `lastSeen` is now the combination of `col` and `id`

However, implementing this approach with a position tag is quite tricky. Let's assume that the data sorting would only look at the record's ID. Then, the upper SQL statement you see next to me would be sufficient. The value for the parameter "last-seen" would be the ID of the record that was transferred last on the current page. In line four of the SQL statement, the question mark is then replaced by the value given in the query. We are looking for all records whose ID is greater than the one transmitted to the server as query parameter "last seen." However, in practice, data is rarely sorted by ID. Often, you want to sort by text or date field. Now, the approach with a position marker becomes a bit tricky for the SQL statement, and the value for the query parameter last-seen becomes much more complicated. In the second SQL statement below, I have indicated the principle. A column "col" must sort the data in addition to the ID. The query parameter "last-seen" value also consists of the value in this column and the ID. The WHERE clause searches for records where the combination of this column and the ID is greater than the specified values. Why is this so complicated? The values can occur several times in the column "col," and only the ID makes the record unique.

20 Summary

- Page based approach
- Offset/size based approach
- Cursor-based approach

So, that's it again for today. Today, you have learned three methods for how the server can divide a significant result set into individual subsets or pages. Which of these three methods you implement depends largely on your taste, especially how you want to offer navigation through the result pages on the client. You have learned about the two problems with data inconsistencies, and you now know how to use the position mark approach to solve them as effectively as possible.

21 Literature

Literature

- Bill Burke: RESTful Java with JAX-RS 2.0: Designing and Developing Distributed Web Services (Englisch). O'Reilly, 2013.